원격 증명 (Remote Attestation)

이 문서는 SEV-SNP의 원격 증명 기능에 대한 상세한 설명을 제공합니다. 원격 증명은 신뢰할 수 없는 환경에서 게스트 VM의 무결성과 신뢰성을 원격으로 검증할 수 있게 해주는 핵심 보안 기능입니다.

목차

- 원격 증명 개념
- 증명 프로세스
- 증명 보고서 생성
- 증명 보고서 검증
- 증명 데이터 구조
- 필드 설명
- 원격 증명 구현 예제
- 보안 고려사항

원격 증명 개념

원격 증명(Remote Attestation)은 신뢰할 수 없는 원격 플랫폼에서 실행되는 소프트웨어의 무결성을 검증하는 보안 메커니즘입니다. SEV-SNP에서 원격 증명은 다음과 같은 보장을 제공합니다:

1. 플랫폼 인증

- **하드웨어 진위성**: 게스트가 진정한 AMD 프로세서에서 실행되고 있음을 증명
- **펌웨어 무결성**: SEV-SNP 펌웨어가 변조되지 않았음을 보장

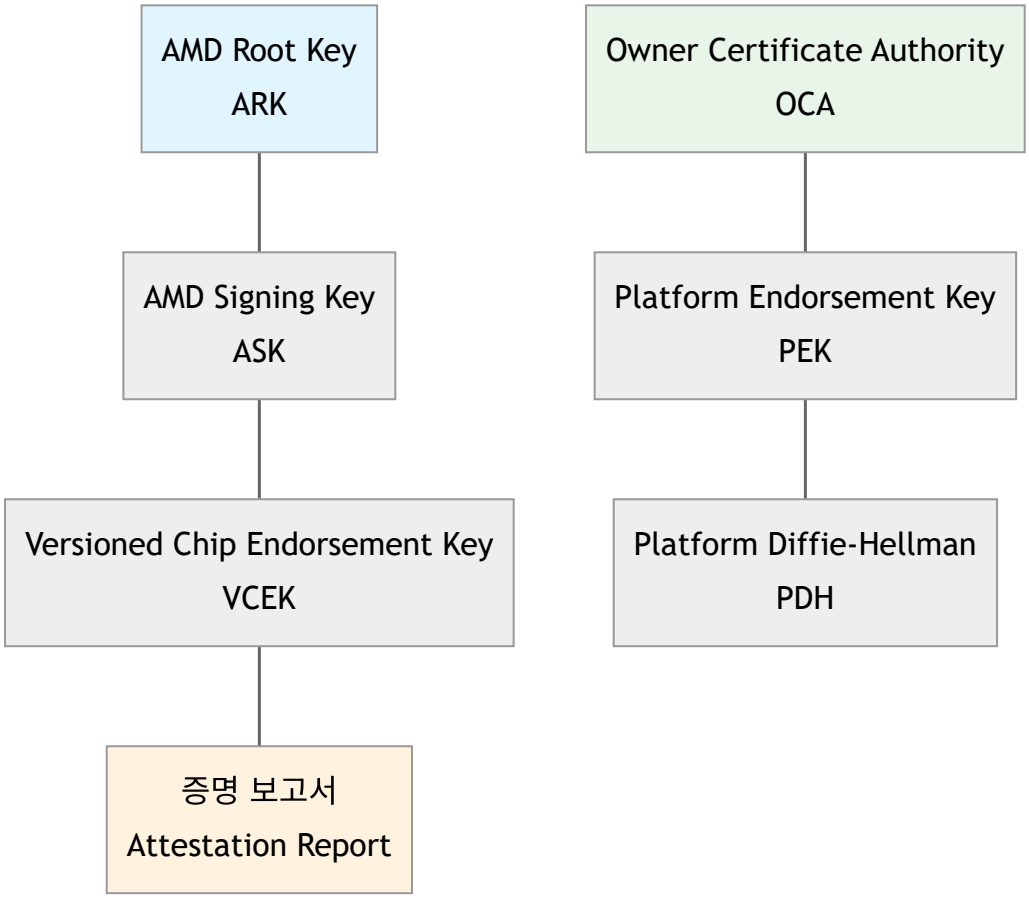
- **플랫폼 소유권**: 플랫폼이 예상된 소유자에 의해 제어되고 있음을 확인

2. 게스트 무결성

- **측정 검증**: 게스트 이미지가 예상된 상태와 일치함을 확인
- **정책 준수**: 게스트가 지정된 보안 정책에 따라 실행되고 있음을 보장
- **런타임 보호**: 게스트가 SEV-SNP의 보호 하에 실행되고 있음을 증명

3. 신뢰 체인

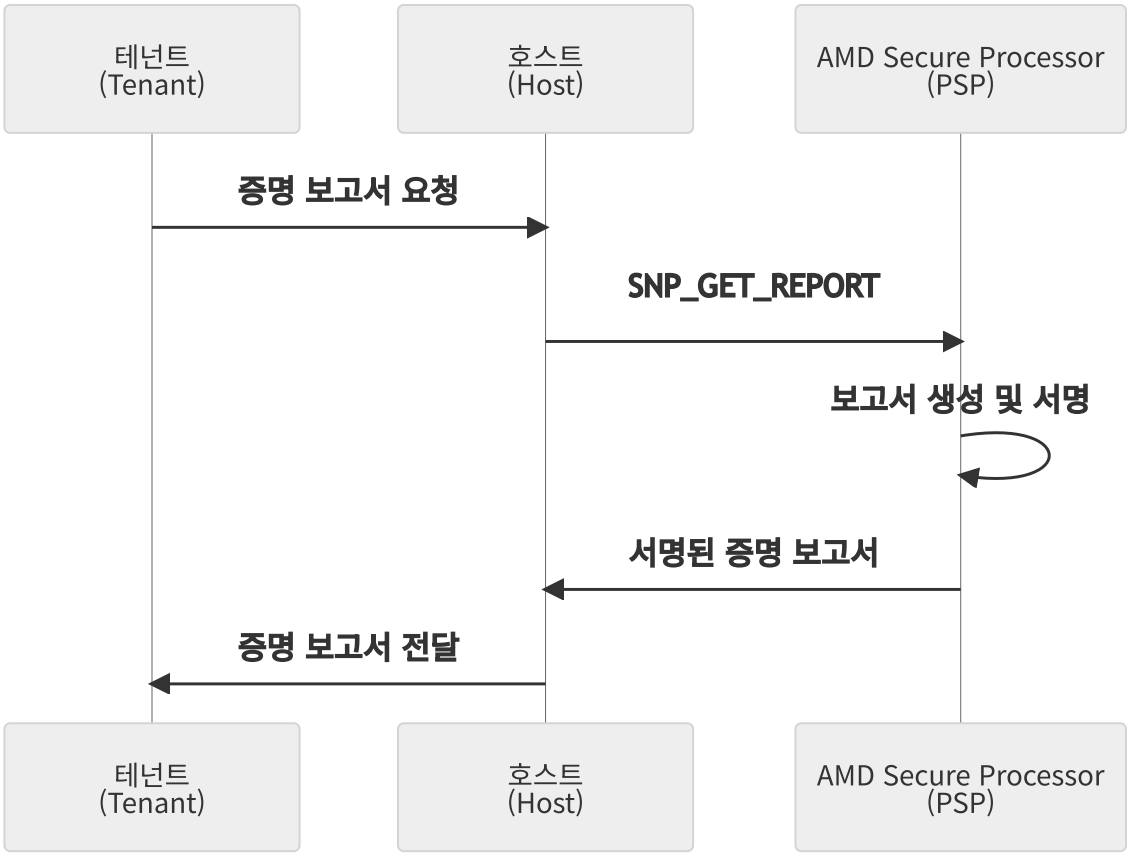
SEV-SNP의 원격 증명은 다음과 같은 신뢰 체인을 기반으로 합니다:



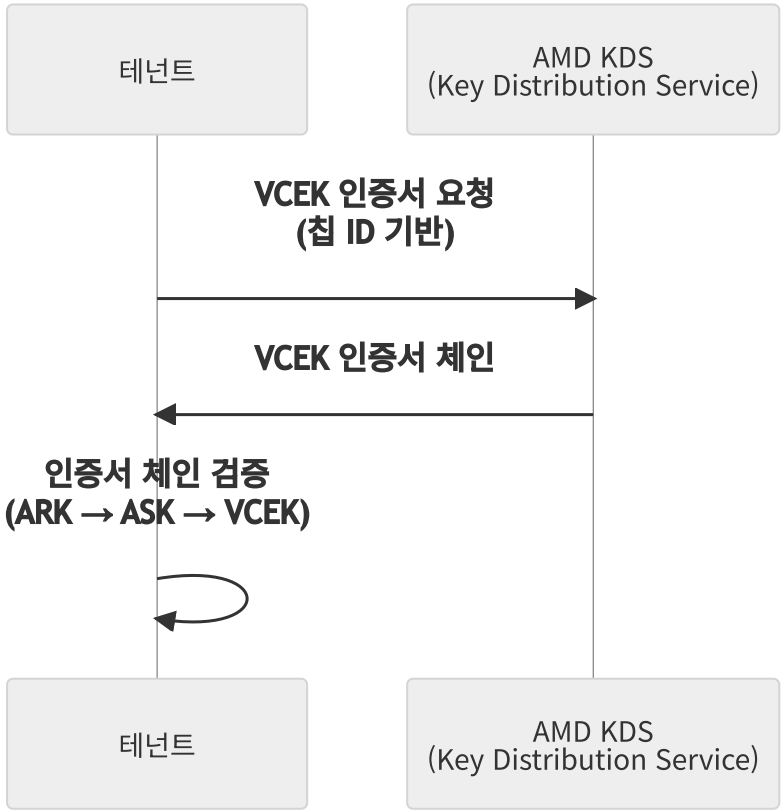
증명 프로세스

SEV-SNP의 원격 증명 프로세스는 다음 단계로 구성됩니다:

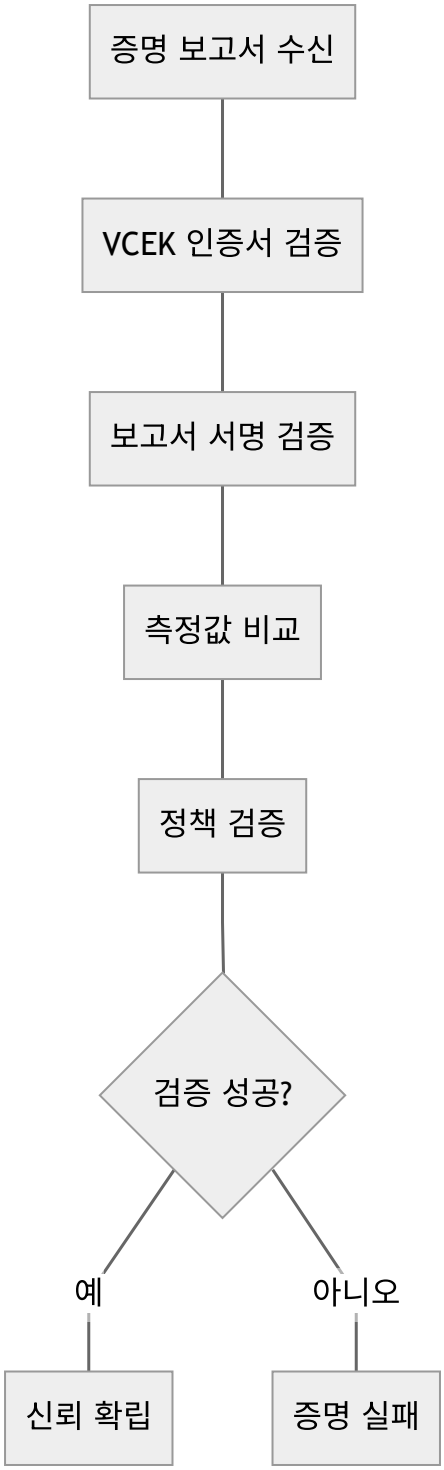
1. 증명 요청 (Attestation Request)



2. 인증서 체인 검증



3. 보고서 검증



증명 보고서 생성

1. 보고서 요청 구조

```
pub struct ReportReq {
/// 증명 보고서에 포함될 게스트 제공 데이터 (64바이트)
```

```
report_data: [u8; 64],
/// 증명 보고서에 포함될 VMPL 레벨
vmpl: u32,
/// 예약된 필드
_reserved: [u8; 28],
}
```

2. 보고서 생성 과정

1. **고유 데이터 준비**: 애플리케이션별 고유 정보 (공개 키, 해시, 논스 등)
2. **VMPL 설정**: 현재 VMPL 레벨 또는 더 높은 레벨 지정
3. **PSP 요청**: AMD Secure Processor에 보고서 생성 요청
4. **서명 생성**: VCEK 개인 키로 보고서에 서명
5. **보고서 반환**: 서명된 증명 보고서 반환

3. 확장 보고서 요청

확장 보고서는 증명 보고서와 함께 인증서 체인을 포함합니다:

```
pub struct ExtReportReq {
/// 기본 보고서 요청
pub data: ReportReq,
/// 인증서 데이터 버퍼 주소
pub certs_address: u64,
/// 인증서 버퍼 길이 (페이지 정렬)
```

```
pub  certs_len: u32,
}
```

증명 보고서 검증

1. 서명 검증 과정

```
// VCEK 인증서로 보고서 서명 검증
impl Verifiable for (&Certificate, &AttestationReport) {
fn  verify(self) -> io::Result<()> {
let  vek = self.0;
let  report = self.1;
// 1. 서명 파싱
let  sig = EcdsaSig::try_from(&report.signature)?;
// 2. 보고서 바이트 직렬화
let  mut  raw_report_bytes = Vec::new();
report.write_bytes(&mut  raw_report_bytes)?;
// 3. 측정 가능한 부분 추출 (0x2a0 바이트까지)
let  measurable_bytes = &raw_report_bytes[..0x2a0];
// 4. SHA-384 해시 계산
let  mut  hasher = Sha384::new();
hasher.update(measurable_bytes);
let  digest = hasher.finish();
```

```
// 5. ECDSA 서명 검증
let ec_key = vek.public_key()?.ec_key()?;
let verified = sig.verify(&digest, &ec_key)?;
match verified {
true => Ok(()),
false => Err(Error::other("VCEK가 증명 보고서에 서명하지 않음")),
}
}
}
```

2. 인증서 체인 검증

```
// 인증서 체인 검증
impl Verifiable for &Chain {
fn verify(self) -> io::Result<Certificate> {
// 1. ARK로 ASK 검증
(&self.ark, &self.ask).verify()?;
// 2. ASK로 VCEK 검증
(&self.ask, &self.vcek).verify()?;
// 3. VCEK 반환
Ok(self.vcek.clone())
}
}
```

증명 데이터 구조

1. 증명 보고서 구조

```
#[repr(C)]
pub struct AttestationReport {
/// 보고서 버전 (현재 2, 3, 5 지원)
pub version: u32,
/// 게스트 보안 버전 번호
pub guest_svn: u32,
/// 게스트 정책
pub policy: GuestPolicy,
/// 런치 시 제공된 패밀리 ID
pub family_id: Array<u8, 16>,
/// 런치 시 제공된 이미지 ID
pub image_id: Array<u8, 16>,
/// 요청된 VMPL 레벨
pub vmpl: u32,
/// 서명 알고리즘 (ECDSA P-384 SHA-384)
pub sig_algo: u32,
/// 현재 TCB 버전
pub current_tcb: TcbVersion,
/// 플랫폼 정보
```

```
pub plat_info: PlatformInfo,
/// 키 정보
pub key_info: KeyInfo,
/// 게스트 제공 데이터 (512비트)
pub report_data: Array<u8, 64>,
/// 런치 시 계산된 측정값
pub measurement: Array<u8, 48>,
/// 하이퍼바이저 제공 데이터
pub host_data: Array<u8, 32>,
/// ID 공개 키의 SHA-384 다이제스트
pub id_key_digest: Array<u8, 48>,
/// 작성자 공개 키의 SHA-384 다이제스트
pub author_key_digest: Array<u8, 48>,
/// 게스트 보고서 ID
pub report_id: Array<u8, 32>,
/// 마이그레이션 에이전트 보고서 ID
pub report_id_ma: Array<u8, 32>,
/// 보고된 TCB 버전
pub reported_tcb: TcbVersion,
/// CPUID 패밀리 ID (버전 3+)
pub cpuid_fam_id: Option<u8>,
/// CPUID 모델 ID (버전 3+)
pub cpuid_mod_id: Option<u8>,
```

```rust
/// CPUID 스테핑 (버전 3+)
pub cpuid_step: Option<u8>,
/// 칩 고유 식별자
pub chip_id: Array<u8, 64>,
/// 커밋된 TCB 버전
pub committed_tcb: TcbVersion,
/// 현재 펌웨어 버전
pub current: Version,
/// 커밋된 펌웨어 버전
pub committed: Version,
/// 런치 시 TCB 버전
pub launch_tcb: TcbVersion,
/// 런치 완화 벡터 (버전 5+)
pub launch_mit_vector: Option<u64>,
/// 현재 완화 벡터 (버전 5+)
pub current_mit_vector: Option<u64>,
/// ECDSA P-384 서명
pub signature: Signature,
}
```

2. 게스트 정책 구조

```rust
bitfield! {
```

```rust
pub struct GuestPolicy(u64);

/// 최소 ABI 마이너 버전

pub abi_minor, set_abi_minor: 7, 0;

/// 최소 ABI 메이저 버전

pub abi_major, set_abi_major: 15, 8;

/// SMT 사용 허용 여부

pub smt_allowed, set_smt_allowed: 16;

/// 마이그레이션 에이전트 허용 여부

pub migrate_ma_allowed, set_migrate_ma_allowed: 18;

/// 디버깅 허용 여부

pub debug_allowed, set_debug_allowed: 19;

/// 단일 소켓 요구 여부

pub single_socket_required, set_single_socket_required: 20;

/// CXL 장치/메모리 허용 여부

pub cxl_allowed, set_cxl_allowed: 21;

/// AES 256 XTS 메모리 암호화 요구 여부

pub mem_aes_256_xts, set_mem_aes_256_xts: 22;

/// RAPL 비활성화 요구 여부

pub rapl_dis, set_rapl_dis: 23;

/// 암호문 숨김 요구 여부

pub ciphertext_hiding, set_ciphertext_hiding: 24;

/// 페이지 스왑 비활성화 여부

pub page_swap_disable, set_page_swap_disable: 25;
```

```
}
```

필드 설명

1. 핵심 필드

- **version**: 증명 보고서 버전 (2, 3, 5)
- **guest_svn**: 게스트 보안 버전 번호, 보안 업데이트 추적용
- **policy**: 게스트 실행 정책, 보안 요구사항 정의
- **measurement**: 게스트 이미지의 SHA-384 해시, 무결성 검증용
- **report_data**: 애플리케이션별 고유 데이터, 논스나 공개 키 포함

2. TCB 관련 필드

- **current_tcb**: 현재 플랫폼의 TCB 버전
- **reported_tcb**: VCEK 생성 시 사용된 TCB 버전
- **committed_tcb**: 플랫폼에 커밋된 TCB 버전
- **launch_tcb**: 게스트 런치 시의 TCB 버전

3. 식별 필드

- **chip_id**: 칩 고유 식별자 (마스킹 가능)
- **family_id**: 게스트 패밀리 식별자
- **image_id**: 게스트 이미지 식별자
- **report_id**: 게스트 보고서 고유 ID

원격 증명 구현 예제

예제 1: 기본 증명 보고서 생성

```
use sev::firmware::guest::Firmware;

fn generate_attestation_report() -> Result<Vec<u8>, Box<dyn std::error::Error>
// 게스트 펌웨어 연결
let mut firmware = Firmware::open()?;
// 고유 데이터 준비 (애플리케이션별 정보)
let mut unique_data = [0u8; 64];
// 예: 공개 키 해시 포함
let public_key_hash = b"my_application_public_key_hash_here_32bytes";
unique_data[..32].copy_from_slice(public_key_hash);
// 예: 타임스탬프 포함
let timestamp = std::time::SystemTime::now()
.duration_since(std::time::UNIX_EPOCH)?
.as_secs();
unique_data[32..40].copy_from_slice(&timestamp.to_le_bytes());
// 증명 보고서 요청
let report = firmware.get_report(
Some(1), // 메시지 버전
Some(unique_data), // 고유 데이터
Some(1), // VMPL 레벨
)?;
```

```
println!("증명 보고서 생성 완료: {} bytes", report.len());

Ok(report)

}
```

예제 2: 확장 증명 보고서 생성

```
use sev::firmware::guest::Firmware;


fn generate_extended_attestation_report() -> Result<(Vec<u8>, Vec<u8>), Box<dyn

let mut firmware = Firmware::open()?;

// 고유 데이터 준비

let unique_data = [42u8; 64]; // 예시 데이터

// 확장 증명 보고서 요청 (인증서 체인 포함)

let (report, cert_chain) = firmware.get_ext_report(

Some(1), // 메시지 버전

Some(unique_data), // 고유 데이터

Some(1), // VMPL 레벨

)?;

println!("확장 증명 보고서 생성 완료:");

println!(" 보고서 크기: {} bytes", report.len());

if let Some(certs) = cert_chain {

println!(" 인증서 개수: {}", certs.len());

for (i, cert) in certs.iter().enumerate() {
```

```
println!(" 인증서 {}: {} bytes", i + 1, cert.data.len());
}
// 인증서 체인을 바이트로 직렬화
let mut cert_bytes = Vec::new();
for cert in certs {
cert_bytes.extend_from_slice(&cert.data);
}
Ok((report, cert_bytes))
} else {
Ok((report, Vec::new()))
}
}
```

예제 3: 증명 보고서 파싱 및 검증

```
use sev::firmware::guest::types::snp::AttestationReport;

fn parse_and_verify_report(report_bytes: &[u8]) -> Result<(), Box<dyn std::err
// 증명 보고서 파싱
let report = AttestationReport::from_bytes(report_bytes)?;
println!("증명 보고서 정보:");
println!(" 버전: {}", report.version);
println!(" 게스트 SVN: {}", report.guest_svn);
```

```
println!(" VMPL: {}", report.vmpl);

println!(" 측정값: {}", hex::encode(&*report.measurement));

// 정책 검증

println!("게스트 정책:");

println!(" ABI 버전: {}.{}", report.policy.abi_major(), report.policy.abi_minor(

println!(" SMT 허용: {}", report.policy.smt_allowed());

println!(" 디버깅 허용: {}", report.policy.debug_allowed());

println!(" 단일 소켓 요구: {}", report.policy.single_socket_required());

// TCB 버전 확인

println!("TCB 정보:");

println!(" 현재 TCB: {}", report.current_tcb);

println!(" 보고된 TCB: {}", report.reported_tcb);

println!(" 커밋된 TCB: {}", report.committed_tcb);

// 보고서 데이터 확인

println!("보고서 데이터: {}", hex::encode(&*report.report_data));

// 서명 정보

println!("서명 알고리즘: {}", report.sig_algo);

println!("서명: {}", report.signature);

Ok(())

}
```

예제 4: 파생 키 생성

```rust
use sev::firmware::guest::{Firmware, DerivedKey, GuestFieldSelect};


fn generate_derived_key() -> Result<[u8; 32], Box<dyn std::error::Error>> {
let mut firmware = Firmware::open()?;
// 파생 키 요청 생성
let mut field_select = GuestFieldSelect::default();
field_select.set_guest_policy(true); // 게스트 정책 포함
field_select.set_measurement(true); // 측정값 포함
field_select.set_tcb_version(true); // TCB 버전 포함
let derived_key_request = DerivedKey::new(
false, // VCEK 사용 (VMRK 아님)
field_select, // 포함할 필드
1, // VMPL 레벨
0, // 게스트 SVN
0, // TCB 버전
Some(0), // 런치 완화 벡터
);
// 파생 키 요청
let derived_key = firmware.get_derived_key(Some(2), derived_key_request)?;
println!("파생 키 생성 완료: {} bytes", derived_key.len());
println!("키: {}", hex::encode(&derived_key));
Ok(derived_key)
```

```
}
```

예제 5: 완전한 증명 워크플로우

```
use sev::firmware::guest::Firmware;
use sev::firmware::guest::types::snp::AttestationReport;

fn complete_attestation_workflow() -> Result<(), Box<dyn std::error::Error>> {
println!("=== SEV-SNP 원격 증명 워크플로우 ===");
// 1. 증명 보고서 생성
println!("1. 증명 보고서 생성 중...");
let mut firmware = Firmware::open()?;
let unique_data = b"MyApp-Attestation-Request-2024-v1.0-SecureData-Hash";
let mut report_data = [0u8; 64];
report_data[..unique_data.len()].copy_from_slice(unique_data);
let (report_bytes, cert_chain) = firmware.get_ext_report(
Some(1),
Some(report_data),
Some(1),
)?;
// 2. 보고서 파싱
println!("2. 증명 보고서 파싱 중...");
let report = AttestationReport::from_bytes(&report_bytes)?;
```

```rust
// 3. 기본 검증
println!("3. 기본 검증 수행 중...");
// 버전 확인
if  report.version < 2 {
return Err("지원되지 않는 보고서 버전".into());
}
// 보고서 데이터 확인
if &*report.report_data != &report_data {
return Err("보고서 데이터 불일치".into());
}
// 정책 확인
if  report.policy.debug_allowed() {
println!("경고: 디버깅이 허용된 게스트입니다");
}
// 4. 측정값 검증 (실제 구현에서는 예상 측정값과 비교)
println!("4. 측정값 검증...");
println!(" 게스트 측정값: {}", hex::encode(&*report.measurement));
// 5. TCB 버전 확인
println!("5. TCB 버전 확인...");
println!(" 현재 TCB: {}", report.current_tcb);
println!(" 보고된 TCB: {}", report.reported_tcb);
// 6. 인증서 체인 확인
if  let Some(certs) = cert_chain {
```

```
println!("6. 인증서 체인 확인...");

println!(" 인증서 개수: {}", certs.len());

// 실제 구현에서는 VCEK 인증서로 보고서 서명을 검증해야 함

// 여기서는 간단히 인증서 존재 여부만 확인

}

println!("✅ 원격 증명 워크플로우 완료!");

Ok(())

}
```

보안 고려사항

1. 신뢰 체인 검증

- **인증서 체인**: ARK → ASK → VCEK 체인의 완전한 검증 필요
- **인증서 폐기**: 폐기된 인증서 확인 및 처리
- **시간 검증**: 인증서 유효 기간 확인

2. 측정값 검증

- **예상 측정값**: 신뢰할 수 있는 소스에서 예상 측정값 획득
- **측정 정확성**: 게스트 이미지의 정확한 측정 방법 이해
- **버전 관리**: 다양한 게스트 버전에 대한 측정값 관리

3. 재생 공격 방지

- **논스 사용**: 보고서 데이터에 고유한 논스 포함
- **타임스탬프**: 시간 기반 검증으로 재생 공격 방지

- **세션 바인딩**: 특정 세션에 보고서 바인딩

4. 정책 검증

- **보안 정책**: 게스트 정책이 보안 요구사항을 만족하는지 확인
- **기능 제한**: 불필요한 기능(디버깅, SMT 등) 비활성화 확인
- **TCB 버전**: 최소 요구 TCB 버전 이상인지 확인

5. 키 관리

- **파생 키**: 적절한 키 파생 매개변수 사용
- **키 격리**: 다른 게스트나 애플리케이션과 키 격리
- **키 순환**: 정기적인 키 갱신 및 순환

이러한 원격 증명 기능을 통해 SEV-SNP는 신뢰할 수 없는 클라우드 환경에서도 강력한 보안 보장을 제공하며, 기밀 컴퓨팅의 핵심 요구사항을 충족합니다.